

개발 학습 노트 — 아키텍처 용어 (Rachel용)

2026-07-20 · 강수아 협업 시작 시점 · 오늘 대화에서 나온 개발 개념을 우리 프로젝트에 붙여 정리 목적: 방향을 내가 쥐고 강수아와 대화할 때, 그리고 내가 성장하면서 전체를 유지·수정할 때 필요한 어휘

0. 큰 그림 — 우리 시스템을 "주방"에 비유

- **엔진(core)** = 주방. 재료(정산서 파일)를 받아 요리(정산 결과)를 만든다. 손님·예약·계산서를 모른다. 오직 "이 재료로 이 요리" 만들 뿐.
- **서비스(service)** = 레스토랑 운영. 손님(출판사·작가), 예약(계약), 계산서(정산 확정·명세서), 직원 권한. 주방에 "이거 만들어줘" 주문을 넣고 결과를 받는다.
- **모듈들(modules)** = 나중에 붙는 별관. 마케팅관, 2차저작물관, 아카이빙관. 주방·홀을 어지럽히지 않고 옆에 확장.
- **핵심: 주방은 순수하게, 운영은 그 위에.** 주방이 예약·계산까지 다 하면 주방이 터진다(= "무거워짐").

1. 모노레포 (Monorepo)

- **뜻:** 여러 부분(core·service·modules)을 **하나의 git 저장소**에 모아 관리하는 방식. ↔ 반대는 폴리레포(부분마다 저장소 따로).
- **비유:** 한 건물 안에 층을 나눠 쓰는 것. 층(모듈)은 구분되지만 건물(저장소)은 하나 → 한 번에 둘러보고 고칠 수 있음.
- **우리에게 왜:** 내가 **전체를 한 곳에서 실행·디버그·수정**해야 하니까. 저장소가 흩어지면 전체를 혼자 굴리기 어려움.
`github.com/rachelchoi11/hivemind` 하나에 core/service를 폴더로 나눔.
- **장점:** 한눈에 봄, 함께 커밋, 전체 검색·수정 쉬움. **단점:** 커지면 저장소가 무거워질 수 있음(우리 규모엔 문제 없음).

2. 모듈 / 패키지 / import

- **모듈(module):** 관련 코드를 묶은 파일·폴더 단위. 예: `engine.py` (매칭 모듈).
- **패키지(package):** 모듈을 묶은 상위 폴더. 예: `core/`.
- **import:** 한 코드가 다른 모듈의 함수를 **불러다 쓰는 것**. 지금 `app.py` 가 `import engine` 해서 `engine.match_records(...)` 를 부름 = 같은 프로그램 안에서 직접 호출.
- **우리 현재:** `app.py` (웹) 가 `engine` (도메인)을 import. 이미 도메인/웹이 파일로 나뉘어 있음 = 리팩터링 거의 안 해도 됨.

3. 라이브러리 호출 vs HTTP API (★ 오늘의 핵심 결정)

두 부분(서비스 ↔ 엔진)이 **어떻게 대화하느냐**의 두 방식.

(A) 라이브러리 호출 = in-process (같은 프로그램 안)

- 서비스가 엔진을 **import** 해서 함수처럼 바로 부름. `result = core.settle(df, params)`
- **비유**: 같은 주방에서 옆 요리사에게 "이거 해줘" 하고 바로 받음. 전화 안 걸어도 됨.
- **장점**: 빠름(네트워크 0), 배포 하나, 디버그 쉬움(한 곳에서 다 돌아감) → **내가 전체를 굴리기 좋음**.
- **단점**: 같은 런타임(같은 파이썬 프로세스)에 있어야 함. 엔진이 무거우면 서비스 프로세스도 무거워짐.

(B) HTTP API = 프로세스 분리 (다른 프로그램끼리)

- 서비스가 엔진에게 **네트워크로 요청**을 보냄. "POST /settle 이 데이터로" → 엔진이 응답으로 결과 돌려줌.
- **비유**: 엔진을 다른 건물(독립 주방)에 두고, 주문을 **전화·택배**로 주고받음.
- **장점**: 엔진을 서비스와 **떼어놓아** 서비스가 가벼움. 엔진만 따로 확장·재시작 가능.
- **단점**: 네트워크 계층·인증·버전 관리 필요. 배포 2개. 2인 초기엔 오버(과함).

우리 결정 (자연스럽게)

지금은 (A) 라이브러리로 간다. 나중에 엔진이 무거워지면 (B) HTTP로 뗀다. - 조건: 엔진을 **순수·무상태**로 유지하면, (A)→(B) 전환이 "리라이트"가 아니라 "걸을 HTTP로 감싸기(래핑)"라서 쉬움. 그래서 지금 못 박지 않아도 나중에 자유롭게 바꿀 수 있음.

4. API (Application Programming Interface)

- **뜻**: 프로그램끼리(또는 부분끼리) 대화하는 **약속된 창구**. "이 요청 보내면 이 응답 준다"는 규칙.
- 두 층위:
- **함수 API**: `settle(df, params)` 같은 함수 시그니처. 라이브러리 호출 때 이게 창구.
- **HTTP API(웹 API)**: `GET /api/publishers`, `POST /api/dist_config/save` 같은 주소. 우리 웹앱이 프론트↔백엔드 사이에서 이미 쓰고 있음.
- **REST**: HTTP API를 자원(publishers, authors) 중심으로 설계하는 관례.
- **HTTP 메서드**: `GET` (조회), `POST` (생성·전송), `PUT` (교체), `DELETE` (삭제). 우리 코드의 `@app.get(...)` / `@app.post(...)` 이 이것.

5. 모놀리스 vs 계층 분리 vs 마이크로서비스

- **모놀리스(monolith)**: 모든 게 한 덩어리 프로그램. **우리 현재 배포본**(app.py에 웹+엔진 다). 작을 땐 편하지만 커지면 얽힘.
- **계층 분리(layered)**: 한 프로그램 안에서 **역할별 층**으로 나눔(core / service / web). 우리가 가려는 곳. 모노레포 +라이브러리 호출과 잘 맞음.
- **마이크로서비스**: 각 부분이 **독립 프로그램**으로 HTTP로 통신. 대규모·대인원용. 우리에게 아직 과함(나중 선택지).
- 스펙트럼: 모놀리스 → **계층 분리(우리 목표)** → 마이크로서비스.

6. 순수 함수 / 무상태 (★ 엔진을 이렇게 유지)

- **순수 함수(pure function)**: 같은 입력 → 항상 같은 출력. 바깥 세상을 안 건드림(DB 저장·전역변수 변경 없음). 예: `match_records(records)` 는 레코드 받아 매칭 결과만 냄.
- **무상태(stateless)**: 자기 안에 **기억(상태)을 안 쌓음**. 계약·정산률·누가 로그인했는지 모름. 필요하면 **인자로 받음** (`settle(df, params)` 의 params).
- **왜 중요**: 순수·무상태면 ① 테스트·디버그 쉬움 ② 어디서든(라이브러리든 HTTP든) 재사용 ③ 프로세스 분리가 쉬움 (= 무거워짐 방어).
- **반대 = 상태(state)/in-memory**: 우리 엔진이 지금 세션에 43만 행을 **메모리에 올려둠**(in-memory). 이게 무거움의 원천. 이 무거운 상태를 서비스가 떠안지 않게, 엔진 쪽에 가둬야 함.

7. 계약 / 인터페이스 (두 사람이 만나는 유일한 지점)

- **계약(contract)**: "이 함수는 이런 입력 받아 이런 출력 준다"는 **약속**. 함수 시그니처가 곧 계약.
- **왜 중요**: 강수아(서비스)와 나(엔진)가 **이 계약만 지키면** 서로 안쪽을 몰라도 됨. 서비스는 `settle(df, params)` 가 어떻게 계산하는지 몰라도 결과만 받으면 됨.
- **규칙**: 계약(함수 목록·입출력)을 **문서로 고정**. 바꿀 땐 **둘이 합의**. 이게 협업의 안전벨트.

8. 스냅샷 / freeze / 불변(immutable)

- **스냅샷(snapshot)**: 어느 시점의 데이터를 **사진처럼 찍어 고정**한 것.
- **freeze(동결)**: 월 정산을 **확정**하면 그 시점의 매칭·정산률·세율을 스냅샷으로 얼려서 **더는 안 바뀌게** 함(불변, immutable).
- **왜 필요**: 확정 후에 매칭 규칙·정산률이 바뀌어도 **이미 발행한 명세서 근거가 흔들리면 안 됨**. 본부장 메모의 "월 정산 확정 시기·확정 데이터 수정 불가"가 이것. 설계서의 CalcSnapshot.

9. Git 협업 — 브랜치 / PR / 리뷰 / 머지 (★ 내가 전체를 쥐는 흐름)

- **브랜치(branch)**: 본선(main)에서 갈라낸 **작업 가지**. 강수아가 `feat/...` 브랜치에서 작업 → 본선 안 건드림. (오늘 우리도 `feat/parser-1112...` 브랜치 공유)
- **커밋(commit)**: 변경 저장 단위(스냅샷 + 설명).
- **push / pull**: 내 커밋을 원격(GitHub)에 올림 / 원격 것을 내려받음.
- **PR(Pull Request)**: "내 브랜치를 본선에 합쳐줘"라는 **요청**. 여기에 변경 내용이 다 보임.
- **리뷰(review)**: PR을 읽고 **확인·질문·승인**. → 강수아 작업은 **PR로 올라오고, 내가 리뷰·머지한다**. 이게 내 성장(코드 다 읽음) + 통제(블랙박스 방지) 지점.
- **머지(merge)**: 리뷰 끝난 PR을 본선에 **합침**.
- **흐름**: 강수아 `브랜치 작업` → `push` → `PR` → 나 `리뷰` → `머지` → 배포.

10. DB / ORM / 마이그레이션 / 스키마

- **DB(데이터베이스)**: 데이터를 **영구 저장**하는 곳. 지금 우리는 메모리·JSON 파일에 임시 저장 → 서비스 계층에선 진짜 DB로.
- **스키마(schema)**: DB의 **구조 설계도**(어떤 표에 어떤 칸). 예: 계약표(작가·작품·정산률·기간).
- **ORM**: 파이썬 객체 ↔ DB 표를 **자동 변환**해주는 도구(SQL 직접 안 써도 됨).
- **마이그레이션(migration)**: DB 구조를 **바꾼 이력을 코드로 관리**(표 추가·칸 변경). 읽히는 마이그레이션 = 내가 나중에 이해·수정 가능.
- **내 제약**: 강수아가 DB·ORM을 고르되, **내가 로컬에서 돌리고 고칠 수 있는 표준적인 것**(예: Postgres/SQLite). 특이한 인프라·마법 도구 지양.

11. 멀티테넌트 / RBAC (서비스 계층 개념)

- **멀티테넌트(multitenant)**: 한 시스템을 **여러 고객(출판사)이 나눠 쓰되 데이터는 서로 격리**. 우리 계정 덩스 = 마스터(하이버마인드) > 출판사 > 작가.
- **RBAC(Role-Based Access Control)**: **역할별 권한 관리**. 관리자·출판사·작가가 볼 수 있는 게 다름.

12. 결합도 / 응집도 — "무거워짐"의 원리

- **결합도(coupling)**: 부분끼리 **얼마나 얽혀 있나**. 높으면 하나 고칠 때 다른 게 깨짐 = 나쁨.
- **응집도(cohesion)**: 한 부분이 **한 가지 일에 집중**하나. 높으면 좋음.
- **무거워짐 = 잘못된 결합**: 서비스가 정산 산식을 복제하거나, 엔진이 계약을 알기 시작하면 결합도↑ → 얽혀서 무거워짐. **경계(계약)로 결합을 낮추는 게** 방어책.

13. 우리 스택 용어 (지금 코드에 쓰는 것)

- **FastAPI**: 파이썬 웹 프레임워크. `@app.get/post` 로 API를 만듦. (`app.py`)
- **uvicorn**: FastAPI 앱을 **띄우는 서버**. `uvicorn app:app --port 8731`.
- **pandas / DataFrame(df)**: 표 형태 데이터를 다루는 파이썬 라이브러리. 43만 행 정산 데이터가 df.
- **세션(session)**: 사용자별 임시 작업 공간(메모리). 지금 정산 데이터가 세션에 올라감.
- **Docker**: 앱을 **어디서든 같게 실행**하게 포장(컨테이너). `Dockerfile`.
- **Railway**: 우리 배포 플랫폼(클라우드). main에 push하면 자동 배포.

14. 오늘 결정을 이 용어로 다시 읽기

1. **엔진(core)을 순수·무상태 패키지로 유지** = 6·2·7번. 계약(7)으로만 만남.
2. **모노레포 + 라이브러리 호출로 시작** = 1·3번. 내가 전체를 로컬에서 굴리려고.
3. **나중에 무거워지면 HTTP로 분리** = 3번(A→B). 순수 유지(6) 덕에 래핑으로 쉽게.

4. 계약·마스터·상태·확정(freeze)은 서비스가 = 5·8·10·11번.
5. 강수아 작업은 PR→내 리뷰·머지 = 9번. 블랙박스 방지 + 성장.

15. 회의에서 이 어휘로 한 줄

"엔진은 **순수·무상태**로 두고 **모노레포·라이브러리 호출**로 시작할게요. 계약·마스터·확정은 **서비스 계층**이 갖고요. 무거워지면 **HTTP로 분리**하되 지금은 아니에요. 작업은 **PR로 주시면 제가 리뷰·머지**할게요."

16. 시나리오 — 출판사 온보딩 (멀티테넌트가 실제로 어떻게 도나)

용어를 실제 흐름에 붙여 보기. "출판사를 영업했다 → 그 출판사가 자기 페이지에서 정산·계약을 한다"를 단계로.

- ① **영업 성사 → 테넌트 등록(provision)** — *service* - 마스터 관리자(하이브마인드)가 출판사를 **새 테넌트**로 생성: 출판사명·레이블·담당자·구독(라이트/정식). - 출판사 관리자 계정 발급 → **초대**(이메일). (오늘 강수아 git 초대와 같은 개념)
- ② **로그인 → 자기 공간만 봄(테넌트 격리)** — *service (멀티테넌트+RBAC)* - 모든 데이터에 **tenant_id**가 붙고, 모든 조회가 "내 tenant_id"로 자동 필터 → 남의 출판사 데이터는 절대 안 보임.
- ③ **출판사 페이지의 3메뉴** | 메뉴 | 하는 일 | 소속 | |---|---|---| | 내 정보 | 프로필·지급계정·구독 | *service* | | 정산 관리 | 정산서 업로드→표준화·매칭·작가별 산출·명세서 | **core 계산** + *service* 귀속·저장 | | 계약 관리 | 작가·작품 계약(정산률·공저·선인세·지급처) | *service* (마스터) | - 정산 숫자는 **core가 계산**, "이 출판사 것"으로 묶고 확정(freeze)은 **service**. core는 끝까지 테넌트를 모름.
- ④ **이 시나리오가 정하는 것** - 계정 덩스: 마스터 > 출판사 > 작가 (작가 포털은 다음 확장) - 라이트=정산 관리만 / 정식=계약 관리까지 (2track 상품) - 입력 주체: 계약·정산률은 **출판사가 직접 입력**(최초 csv 일괄 + 신규작 노티)
- ⑤ **두 모델 공존(꼭 구분)** - **Self-serve**: 출판사가 직접 로그인해 자기 걸 관리 (SaaS 상품) - **유통대행**: 우리(스토린랩)가 여러 출판사 걸 대신 운영 → "출판사별 정산" 뷰로 각 몫 확인 (오늘 만든 화면) - 설계상 **"테넌트=출판사"는 동일, 누가 운영하나(출판사 vs 우리)만 권한으로 갈림**. 한 모델로 잡으면 안 꼬임.